Online PES Decoder Learning in Neuromorphic Residual UAV Attitude Control

The nominal controller families of Sec. VI-B and a recursive backstepping design: equations, derivations, code, and architectures
companion to the ACC 2027 manuscript (version of September 30, 2026)

Gandhi Contreras, Steven Sandoval, and Luis Rodolfo Garcia Carrillo

October 1, 2026

Abstract

Section VI-B of the manuscript attaches the same gated CMG–PES residual to six nominal attitude controllers: PD with desired-rate feedforward in backstepping form (the primary baseline), PID with rate feedforward, LQR, sliding-mode control (SMC), finite-horizon model predictive control (MPC), and grid-search nonlinear MPC (NMPC). This document gives, for each family, its architecture, its control law, the derivation of its gains, the exact lines of code that implement it, and the settings used in the portability study (Amendment A5). All six families share one torque interface and are analyzed in one coordinate, the rotation-vector error $\boldsymbol{e} = 2\boldsymbol{e}_q$, so their small-error behavior can be compared directly (Table 2): every family reduces near the reference to a second-order loop at the same bandwidth scale, with damping ratios from 0.71 (LQR, MPC) to 1 (SMC). Each gain is read from the class the runs used, and each closed-form derivation is checked numerically. The family the code calls "backstepping" is computed torque; Section 4 therefore adds a recursive Lyapunov backstepping design (plain and adaptive) in the style of the EHEKATL BWB controllers, with its stability derivation, numerical verification, and a 40-run campaign with and without the residual. This new family is not part of the manuscript. The source files cited here are byte-identical to the ones recorded by every A5 run (60/60 hashes).

Contents

1 How to read this document **3**

2 What all six families share **3**
2.1 The residual attaches to any nominal torque . . . 3
2.2 Error coordinates and the design model . . . 4
2.3 Torque box and platform constants . . . 5
2.4 The A5 settings . . . 5
2.5 Small-error equivalence of the six families . . . 6

3 PD with desired-rate feedforward in computed-torque form (primary baseline) **7**

4 Recursive Lyapunov backstepping (new family) **8**
4.1 Strict-feedback form . . . 9
4.2 Step 1: kinematic subsystem . . . 9
4.3 Step 2: rate subsystem . . . 9
4.4 Gain design: same linearization as PD+ff . . . 10
4.5 Verification . . . 10

4.6 Results: recursive backstepping with and without the residual 11

5 PID with rate feedforward and rotor-level anti-windup 13

6 LQR 14

7 Sliding-mode control 15

8 Model predictive control (unconstrained, finite horizon) 16

9 Nonlinear MPC (grid search around the backstepping law) 18

10 Results across families (Sec. VI-B) 20

11 Scope of the family comparison 21

A Source files and hashes 21

1 How to read this document

- **Paper references.** "P-(15)" is equation (15) of the manuscript; Sec. VI-B and Proposition 1 refer to the manuscript. The equation-to-code companion of September 30 (`paper/companion_2026-09-30`) covers the quaternion conventions, the residual and the gate in detail; this document does not repeat them.
- **Code.** Every listing is read from the repository file at the quoted lines, with the file's own line numbers. Four files carry all six families: `experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py` (class `AdvancedBaseline`: backstepping, LQR, SMC, MPC, NMPC), `experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py` (the A5 plant loop and the PID path), and the campaign drivers `run_campaign_a5.py` and `run_campaign_a5_nmpc_corrected.py`. Appendix A gives their hashes.
- **Status tags.** *Standard*: textbook result, cited. *Derived here*: the steps are written out. *Verified*: recomputed by `tools/build_family_tables.py` from the code itself. *Implementation note*: a detail of the code that the paper leaves implicit, with its effect on reported numbers.
- **Figures.** Every architecture uses the manuscript's figure style. The left part (reference and attitude error) and the right part (torque box, gated residual, allocation, plant) are identical in all six figures; the dashed blue group is the only part that changes from family to family.

2 What all six families share

2.1 The residual attaches to any nominal torque

The residual reads only the nominal request $\boldsymbol{u}_0$ and adds a bounded, gated correction to it (Fig. 1). Proposition 1 assumes only $\boldsymbol{u}_{0,k} \in \mathcal{U} = \{\boldsymbol{u} : |u_i| \le \ell_i\}$, so it holds for every family whose output is saturated to the torque box. All six are: LQR, backstepping, SMC and MPC through the shared clip (line 241 of `experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`), NMPC by clipping every candidate (lines 265 and 273), and PID at line 353 of `experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py`. Exact recovery at $\beta = 0$ was certified by running the augmentor with $\beta = 0$ against the classical run: relative difference 0 in all five cases (LQR, SMC, MPC and two NMPC missions; `certify_beta0_recovery.py`). Backstepping is covered by the 20/20 bit-exact match with the sealed primary runs; PID has no separate $\beta = 0$ run, and for it the recovery rests on Proposition 1 alone.

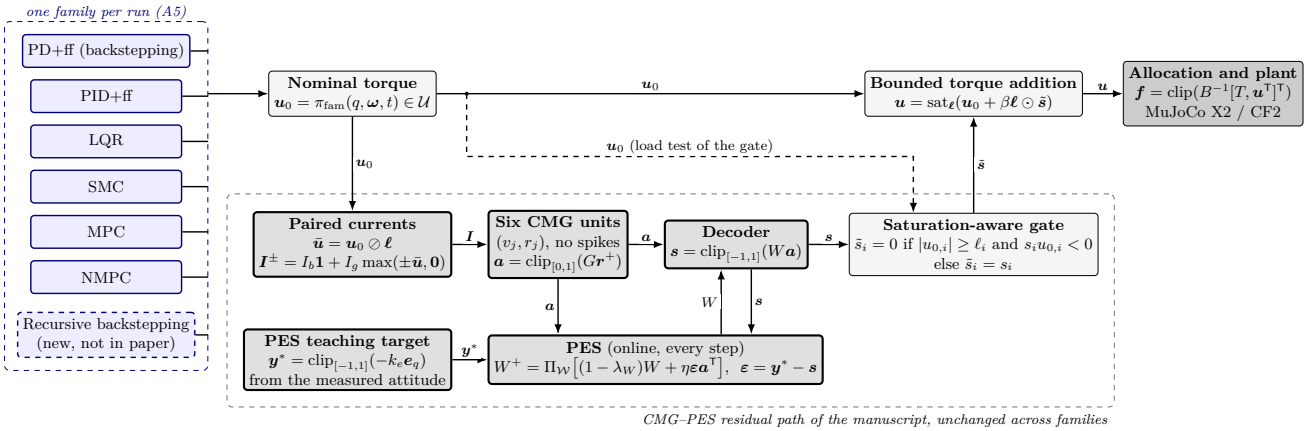


Figure 1: One residual, six nominal controllers. The nominal torque comes from one family per run; the residual path is the manuscript's Fig. 1 unchanged ($\beta = .5$ on X2, .2 on CF2; $\eta = 8 \times 10^{-4}$; $k_e = 8$). The gate and the outer saturation see only $\boldsymbol{u}_0$, so nothing in the residual depends on how $\boldsymbol{u}_0$ was computed.

2.2 Error coordinates and the design model

All families act on the shortest-path error $\boldsymbol{e}_q = \text{sgn}_+(q_{e,0}) \boldsymbol{q}_{e,v}$ (P-(4)–P-(6)) and the rate error $\boldsymbol{\omega}_e = \boldsymbol{\omega} - \boldsymbol{\omega}_d$, with $\boldsymbol{\omega}_d$ the central-difference reference rate of the companion, Sec. 2.5. `AdvancedBaseline` works in the rotation-vector coordinate

$$\boldsymbol{e} = 2\boldsymbol{e}_q, \qquad \|\boldsymbol{e}\| = 2 \sin \tfrac{\theta_q}{2} = \theta_q + O(\theta_q^3), \tag{1}$$

and designs every law for the per-axis double integrator obtained near the reference. From the error kinematics P-(7)–P-(8), $\dot{\boldsymbol{e}} = q_{e,0} \boldsymbol{\omega}_e + \boldsymbol{e}_q \times \boldsymbol{\omega}_e \approx \boldsymbol{\omega}_e$; from Euler's equation P-(2) with the principal moments $\boldsymbol{J} = \boldsymbol{j}_{\text{tune}}$ (Table I of the paper),

$$\dot{e}_i \approx \omega_{e,i}, \qquad \dot{\omega}_{e,i} = a_i - \dot{\omega}_{d,i} - \left[(\boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}) \oslash \boldsymbol{J}\right]_i, \qquad a_i = \tau_i / J_i. \tag{2}$$

Every family outputs a desired angular acceleration $\boldsymbol{a}$; the torque is $\boldsymbol{\tau} = \boldsymbol{J} \odot \boldsymbol{a}$ plus, optionally, the gyroscopic term, saturated to the box:

$$\boldsymbol{u}_0 = \text{sat}_{\boldsymbol{\ell}}\left(\boldsymbol{J} \odot \boldsymbol{a} + \chi_{\text{ff}}\, \boldsymbol{\omega} \times (\boldsymbol{J} \odot \boldsymbol{\omega})\right), \qquad \boldsymbol{a} = \chi_{\text{ff}} \dot{\boldsymbol{\omega}}_d + \boldsymbol{a}_{\text{fam}}(\boldsymbol{e}, \boldsymbol{\omega}_e). \tag{3}$$

Derived here. In every run of the paper $\chi_{\text{ff}} = 0$ (`--no-model-ff`): the acceleration feedforward and the gyroscopic term are off, so each family is a feedback law on $(\boldsymbol{e}, \boldsymbol{\omega}_e)$ scaled by inertia, and the residual is the only adaptive element. The rate feedforward $\boldsymbol{\omega}_d$ stays on (`--rate-ff left-neg`) for all families.

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 189–199 $\boldsymbol{\omega}_d$ and $\dot{\boldsymbol{\omega}}_d$ (the latter is zero when model feedforward is off)

```
189     # ---- references ----------------------------------------------------------
190     def _wd(self, t):
191         if self.mode == "none":
192             return np.zeros(3)
193         return reference_rate(t, self.a.mission, self.a.duration, self.mode)
194
195     def _wd_dot(self, t, h=1e-3):
196         if self.mode == "none" or not self.model_ff:
197             return np.zeros(3)
198         t0 = max(t - h, 0.0)
199         return (self._wd(t + h) - self._wd(t0)) / ((t + h) - t0)
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 216–245 `AdvancedBaseline.torque`: one interface, five families, shared clip at line 241

```python
    # ---- the control law ------------------------------------------------------
    def torque(self, t, eq, omega, q_meas):
        if self.hold is not None and self.count % self.every != 0:
            self.count += 1
            return self.hold
        self.count += 1
        # NMPC self-check: compare last update's one-period-ahead prediction
        # with where the attitude actually is now. A convention error in the
        # propagation shows up as prediction error of the size of the motion.
        if self.pred is not None:
            self.pred_err.append(quat_angle(quat_error(self.pred, q_meas)[0]))
            self.hold_err.append(quat_angle(quat_error(self.q_prev, q_meas)[0]))
        wd, wd_dot = self._wd(t), self._wd_dot(t)
        e, we = 2.0 * eq, omega - wd
        gyro = np.cross(omega, self.J * omega) if self.model_ff else np.zeros(3)
        b = self.a.baseline
        if b in ("lqr", "backstepping"):
            acc = wd_dot - self.k1 * e - self.k2 * we
        elif b == "smc":
            s_ = we + self.lam * e
            acc = wd_dot - self.lam * we - self.eta * np.clip(s_ / self.phi, -1.0, 1.0)
        elif b == "mpc":
            acc = wd_dot - (self.K0[:, 0] * e + self.K0[:, 1] * we)
        elif b == "nmpc":
            acc = wd_dot - self.k1 * e - self.k2 * we
        tau = np.clip(self.J * acc + gyro, -self.lim, self.lim)
        if b == "nmpc":
            tau = self._nmpc(t, q_meas, omega, tau)
        self.hold = tau
        return tau
```

2.3 Torque box and platform constants

$\ell_i = J_i \nu_i$ with $\boldsymbol{\nu} = (12, 12, 10)\,\mathrm{rad/s^2}$, the same for both platforms, so the box limits every family to the same angular acceleration $|a_i| \le \nu_i$:

Table 1: Constants read from the A5 run manifests (`derived` field).

	X2	CF2 (1.5× rotor design)
$\boldsymbol{J}$ (kg m^2)	0.06071, 0.03647, 0.02541	2.395e-05, 2.395e-05, 3.235e-05
$\boldsymbol{\ell}$ (N m)	0.7285, 0.4376, 0.2541	0.0002874, 0.0002874, 0.0003235
$\boldsymbol{\ell} \oslash \boldsymbol{J}$ (rad/s^2)	12.00, 12.00, 10.00	12.00, 12.00, 10.00
$\rho\boldsymbol{\omega}_n$ (rad/s)	12, 12, 10.5	12, 12, 10.5

Because the box is defined as an acceleration, every gain in this document is the same for X2 and CF2 (the script asserts it), and the torque differs only by the inertia factor.

2.4 The A5 settings

All six families ran with the same plant, missions, residual, gate, $\rho = 3$ bandwidth scale and seed; only the family flags differ:

`experiments/final_robustness_gating_2026-09-20/run_campaign_a5.py` : 27–40 per-family flags

```
FAMILY_ARGS = {
    "lqr": [],
    "smc": ["--smc-eta-frac", "0.5"],
    "mpc": ["--mpc-dt", "0.02", "--mpc-horizon", "120", "--mpc-q-rate", "0.0"],
    "nmpc": ["--nmpc-dt", "0.06", "--nmpc-horizon", "5", "--nmpc-grid", "3", "--nmpc-predict", "policy",
             "--nmpc-offset-frac", "0.1", "--nmpc-rate-weight", "2.0", "--baseline-rate-hz", "100"],
    "pid": ["--baseline", "pd", "--ki-factor", "0.3", "--antiwindup", "rotor"],
}


def family_flag(family: str) -> list[str]:
    if family == "pid":
        return FAMILY_ARGS["pid"]
    return ["--baseline", family] + FAMILY_ARGS[family]
```

`experiments/final_robustness_gating_2026-09-20/run_campaign_a5.py` : 49–56 common flags: $\rho = 3$, rate feedforward on, model feedforward off, gate on in the CMG arm

```
    cmd = [sys.executable, str(RUNNER), "--platform", platform, "--controller", controller, "--mission", mission
     ,
           "--duration", "45.0", "--dt", "0.0005", "--beta", str(beta), "--learning-rate", "0.0008",
           "--k-error", "8", "--k-rate", "0", "--actuator-scale", str(ACTUATOR_SCALE[platform]), "--seed", "1",
           "--bw-scale", "3.0", "--rate-ff", "left-neg", "--no-model-ff",
           "--log-every", "10", "--no-neural-archive", "--output", str(out)] + family_flag(family)
    if controller == "nengo-nef-pes":
        cmd += ["--output-gate", "torque-saturation-opposition", "--gate-load-threshold", "1.0",
                "--gate-epsilon-fraction", "0", "--gate-epsilon-nm", "1e-12"]
```

Implementation note. The NMPC row of the paper comes from `run_campaign_a5_nmpc_corrected.py`, which drops the `--baseline-rate-hz 100` hold of the first A5 run so that NMPC, like every other family, updates at every 0.5-ms step (Sec. 9). The held run is kept on disk and not used.

2.5 Small-error equivalence of the six families

Near the reference, inside the SMC boundary layer and away from the box, every family reduces to $a_i = -k_{1,i}e_i - k_{2,i}\omega_{e,i}\,(-k_{I,i}\int e_i)$, i.e. the characteristic polynomial $s^2 + k_2 s + k_1$ (or $s^3 + k_2 s^2 + k_1 s + k_I$ for PID). Table 2 lists the gains each family actually uses; Table 3 gives the resulting poles of the linearized design model (2).

Table 2: Equivalent small-error gains, read from `AdvancedBaseline` with the A5 settings (roll / yaw; pitch equals roll). $\omega_{\text{eq}} = \sqrt{k_1}$, $\zeta_{\text{eq}} = k_2/(2\sqrt{k_1})$. "Box at" is the rotation angle at which the proportional term alone reaches the torque box, $\nu_i/k_{1,i}$. *Verified.*

Family	k_1 (s^{-2})	k_2 (s^{-1})	ω_{eq} (rad/s)	ζ_{eq}	box at (°)	k_I (s^{-3})
PD+ff (backstepping form)	144.0/110.2	22.80/19.95	12.00/10.50	0.950/0.950	4.77/5.20	–
PID+ff	144.0/110.2	22.80/19.95	12.00/10.50	0.950/0.950	4.77/5.20	43.2/33.1
LQR	144.0/110.2	16.97/14.85	12.00/10.50	0.707/0.707	4.77/5.20	–
SMC	144.0/110.2	24.00/21.00	12.00/10.50	1.000/1.000	4.77/5.20	–
MPC	121.5/95.0	15.59/13.79	11.02/9.75	0.707/0.707	5.66/6.03	–
NMPC	144.0/110.2	22.80/19.95	12.00/10.50	0.950/0.950	4.77/5.20	–

Table 3: Closed-loop poles of the linearized design model (2) with each family's gains. *Verified.*

Family	axis	poles (s^{-1})
PD+ff (backstepping form)	roll/pitch	$-11.40 \pm 3.75j$
PD+ff (backstepping form)	yaw	$-9.98 \pm 3.28j$
PID+ff	roll/pitch	$-11.24 \pm 3.24j$, -0.32
PID+ff	yaw	$-9.82 \pm 2.77j$, -0.32
LQR	roll/pitch	$-8.49 \pm 8.49j$
LQR	yaw	$-7.42 \pm 7.42j$
SMC	roll/pitch	-12.00 (double)
SMC	yaw	-10.50 (double)
MPC	roll/pitch	$-7.80 \pm 7.80j$
MPC	yaw	$-6.89 \pm 6.89j$
NMPC	roll/pitch	$-11.40 \pm 3.75j$
NMPC	yaw	$-9.98 \pm 3.28j$

The families therefore differ in damping and in what happens away from the origin: how each one behaves at the box (SMC uses a bounded reaching term, PID freezes its integrator, NMPC searches around the nominal law), not in bandwidth. The proportional term of every linear law reaches the box at a rotation error of 5–6°, so Hardstep's steps of tens of degrees saturate every family, which is where the gate matters.

3 PD with desired-rate feedforward in computed-torque form (primary baseline)

In the code this family is called `backstepping` (`--baseline backstepping`); it is the computed-torque form. It is not a recursive backstepping design; the recursive design is Sec. 4.

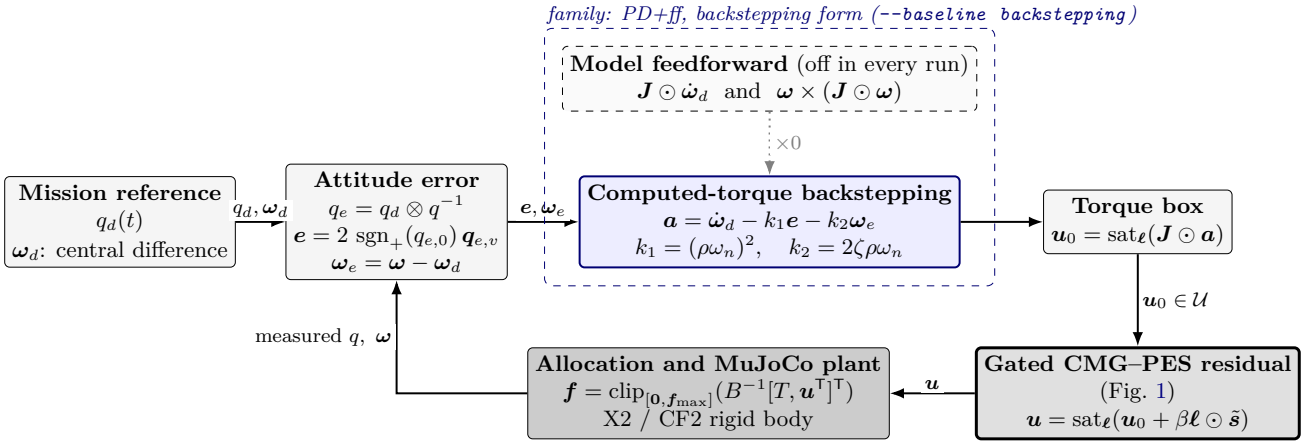


Figure 2: PD+ff in computed-torque (backstepping) form, the paper's $\rho = 3$ primary baseline. The model feedforward path (dotted) exists in the code and is switched off in every run, which makes this law identical to P-(15).

Law. Computed-torque backstepping [1, 2]: choose the error dynamics $\dot{\omega}_e = -k_1 e - k_2 \omega_e$ and invert (2),

$$u_0 = \mathrm{sat}_\ell \Big(J \odot (\dot{\omega}_d - k_1 e - k_2 \omega_e) + \omega \times (J \odot \omega) \Big), \qquad k_1 = (\rho\omega_n)^2, \quad k_2 = 2\zeta\rho\omega_n, \ \zeta = 0.95. \tag{4}$$

With the feedforward on, no saturation and a diagonal inertia, the gyroscopic term and $\dot{\boldsymbol{\omega}}_d$ cancel and the rate error obeys $\dot{\boldsymbol{\omega}}_e = -k_1\boldsymbol{e} - k_2\boldsymbol{\omega}_e$ exactly; with $\dot{\boldsymbol{e}} \approx \boldsymbol{\omega}_e$ this is $\ddot{e}_i + k_2\dot{e}_i + k_1 e_i \approx 0$ near the reference, the textbook reason for this form. *Standard.*

Reduction to the paper's PD+ff. With $\chi_{\text{ff}} = 0$ and $\boldsymbol{e} = 2\boldsymbol{e}_q$, $\boldsymbol{u}_0 = \text{sat}_{\boldsymbol{\ell}}(-2\boldsymbol{J}(\rho\boldsymbol{\omega}_n)^2 \odot \boldsymbol{e}_q - 2\zeta\boldsymbol{J}\rho\boldsymbol{\omega}_n \odot \boldsymbol{\omega}_e)$, which is P-(13)–P-(15) with $\boldsymbol{\omega}_n \mapsto \rho\boldsymbol{\omega}_n$. *Derived here; verified bit-exact in 20/20 cells* (`run_campaign_a5_backstepping.py`, relative deviation 0 against the sealed primary runs).

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 138–141 gains $k_1 = \omega^2$, $k_2 = 2\zeta\omega$, $\omega = \rho\omega_n$

```
        elif b in ("backstepping", "nmpc"):
            # Computed-torque backstepping (aero2 form): desired accel
            # -k1 e - k2 w_e, then inverse dynamics. NMPC uses it as nominal.
            self.k1, self.k2 = self.wn ** 2, 2.0 * self.zeta * self.wn
```

`experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py`: 79–86 the frozen paper baseline: backstepping, $\rho = 3$, $\zeta = .95$, `left-neg`, no model feedforward

```
# Frozen paper-facing baseline.  AdvancedBaseline with backstepping and
# --no-model-ff is algebraically PD + desired-rate feedforward.
BASE_WN_RAD_S = np.array([4.0, 4.0, 3.5])
BASELINE_RHO = 3.0
BASELINE_ZETA = 0.95
BASELINE_FAMILY = "backstepping"
BASELINE_RATE_FF = "left-neg"
BASELINE_MODEL_FF = False
```

4 Recursive Lyapunov backstepping (new family)

The family of Sec. 3 is computed torque: it chooses linear error dynamics and inverts the model, and with model feedforward off it is exactly PD+ff. This section adds a backstepping controller in the strict sense: a recursive design with a virtual control on the kinematic subsystem, a cross-coupling term that cancels the interconnection in the Lyapunov derivative, and a final Lyapunov function that certifies the closed loop [1, 3]. It follows step by step the design used for the longitudinal and adaptive controllers of the EHEKATL BWB (strict-feedback form, errors z_1, z_2, virtual control with the term $-g_1^{\mathsf{T}} z_1$, $V = \sum \frac{1}{2} z_i^{\mathsf{T}} z_i$, La Salle–Yoshizawa, adaptive estimates with $\frac{1}{2}\tilde{f}^{\mathsf{T}}\Gamma\tilde{f}$). The plain and the adaptive versions are implemented in `experiments/backstepping_true_2026-10-01/true_backstepping.py` and run through the unchanged A5 runner. **They are not part of the manuscript.**

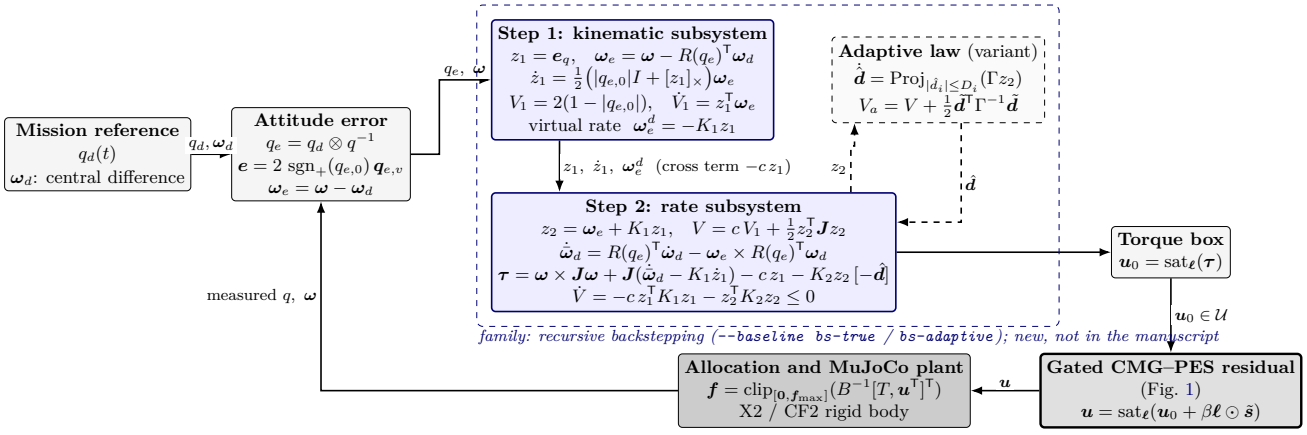


Figure 3: Recursive backstepping. Step 1 stabilizes the quaternion kinematics with a virtual rate; step 2 drives the rate to it with a torque that cancels the gyroscopic term, feeds forward the transported desired-rate derivative, and contains the coupling term $-c\,z_1$. The adaptive variant adds a projected estimate of an unknown body torque.

4.1 Strict-feedback form

With the passive convention of the paper, $q_e = q_d \otimes q^{-1}$ and its body-frame rate $\boldsymbol{\omega}_e = \boldsymbol{\omega} - \bar{\boldsymbol{\omega}}_d$, $\bar{\boldsymbol{\omega}}_d = R(q_e)^{\mathsf{T}}\boldsymbol{\omega}_d$ (P-(7)–P-(8), with the transport $R(q_e)^{\mathsf{T}}$ kept), the attitude system is in strict-feedback form:

$$\dot{z}_1 = \tfrac{1}{2}T(q_e)\,(\boldsymbol{\omega}-\bar{\boldsymbol{\omega}}_d), \qquad \boldsymbol{J}\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}\times\boldsymbol{J}\boldsymbol{\omega}+\boldsymbol{\tau}+\boldsymbol{d}, \qquad z_1 = \boldsymbol{e}_q = \operatorname{sgn}_+(q_{e,0})\boldsymbol{q}_{e,v}, \quad T(q_e) = |q_{e,0}|I+[z_1]_\times, \tag{5}$$

i.e. $x_1 = z_1$, $x_2 = \boldsymbol{\omega}$, $f_1 = -\frac{1}{2}T\bar{\boldsymbol{\omega}}_d$, $g_1 = \frac{1}{2}T$, $f_2 = -\boldsymbol{J}^{-1}(\boldsymbol{\omega}\times\boldsymbol{J}\boldsymbol{\omega})$, $g_2 = \boldsymbol{J}^{-1}$ in the thesis notation, with $\boldsymbol{d}$ an unknown body torque (zero in the plain design). There is no actuator state: the rotor forces of the MuJoCo models act without lag, so the thesis's z_3 step is not needed. The first identity follows from P-(8): $\frac{d}{dt}(\operatorname{sgn}_+(q_{e,0})\boldsymbol{q}_{e,v}) = \frac{1}{2}(|q_{e,0}|\boldsymbol{\omega}_e + z_1\times\boldsymbol{\omega}_e)$.

4.2 Step 1: kinematic subsystem

Take $V_1 = 2(1-|q_{e,0}|) \geq 0$, zero exactly at $q_e = \pm 1$. With $\dot{q}_{e,0} = -\frac{1}{2}\boldsymbol{q}_{e,v}^{\mathsf{T}}\boldsymbol{\omega}_e$ (P-(8)),

$$\dot{V}_1 = -2\,\operatorname{sgn}_+(q_{e,0})\,\dot{q}_{e,0} = z_1^{\mathsf{T}}\boldsymbol{\omega}_e.$$

Treat $\boldsymbol{\omega}_e$ as the control of this subsystem and choose the virtual rate $\boldsymbol{\omega}_e^d = -K_1 z_1$, $K_1 = \operatorname{diag}(k_{1,i}) > 0$: if $\boldsymbol{\omega}_e = \boldsymbol{\omega}_e^d$, then $\dot{V}_1 = -z_1^{\mathsf{T}}K_1 z_1$.

4.3 Step 2: rate subsystem

Define the second error $z_2 = \boldsymbol{\omega}_e - \boldsymbol{\omega}_e^d = \boldsymbol{\omega}_e + K_1 z_1$ and $V = c\,V_1 + \frac{1}{2}z_2^{\mathsf{T}}\boldsymbol{J}z_2$, $c > 0$. Using $\boldsymbol{\omega}_e = z_2 - K_1 z_1$ and $\dot{\boldsymbol{\omega}}_e = \dot{\boldsymbol{\omega}} - \dot{\bar{\boldsymbol{\omega}}}_d$,

$$\dot{V} = c\,z_1^{\mathsf{T}}z_2 - c\,z_1^{\mathsf{T}}K_1 z_1 + z_2^{\mathsf{T}}\big(-\boldsymbol{\omega}\times\boldsymbol{J}\boldsymbol{\omega} + \boldsymbol{\tau} + \boldsymbol{d} - \boldsymbol{J}\dot{\bar{\boldsymbol{\omega}}}_d + \boldsymbol{J}K_1\dot{z}_1\big).$$

The control

$$\boxed{\boldsymbol{\tau} = \boldsymbol{\omega}\times\boldsymbol{J}\boldsymbol{\omega} + \boldsymbol{J}(\dot{\bar{\boldsymbol{\omega}}}_d - K_1\dot{z}_1) - c\,z_1 - K_2 z_2 - \hat{\boldsymbol{d}}}, \qquad \boldsymbol{u}_0 = \operatorname{sat}_{\boldsymbol{\ell}}(\boldsymbol{\tau}), \tag{6}$$

with $K_2 = \operatorname{diag}(k_{2,i}) > 0$ and $\dot{z}_1 = \frac{1}{2}T(q_e)\boldsymbol{\omega}_e$ computed from measured signals, gives

$$\dot{V} = -c\,z_1^{\mathsf{T}}K_1 z_1 - z_2^{\mathsf{T}}K_2 z_2 + z_2^{\mathsf{T}}\tilde{\boldsymbol{d}}, \qquad \tilde{\boldsymbol{d}} = \boldsymbol{d} - \hat{\boldsymbol{d}}. \tag{7}$$

The term $-c\,z_1$ is the backstepping coupling term: it cancels $c\,z_1^{\mathsf{T}}z_2$, the interconnection between the two subsystems ($-g_1^{\mathsf{T}}z_1$ in the thesis). Without it the design is computed torque.

Plain law ($\boldsymbol{d} = \hat{\boldsymbol{d}} = \boldsymbol{0}$). $\dot{V} = -c\,z_1^{\mathsf{T}}K_1 z_1 - z_2^{\mathsf{T}}K_2 z_2 < 0$ for $(z_1, z_2) \neq 0$; by La Salle–Yoshizawa $z_1, z_2 \to 0$, hence $q_e \to \pm 1$ and $\boldsymbol{\omega} \to \bar{\boldsymbol{\omega}}_d$, and $z_1, z_2 \in \mathcal{L}_2$. Because $c\,V_1 \geq c\|z_1\|^2/2$ near the origin, the convergence is exponential there.

Adaptive law. For a constant unknown $\boldsymbol{d}$, add $\frac{1}{2}\tilde{\boldsymbol{d}}^{\mathsf{T}}\Gamma^{-1}\tilde{\boldsymbol{d}}$ to V. Since $\dot{\tilde{\boldsymbol{d}}} = -\dot{\hat{\boldsymbol{d}}}$, the update

$$\dot{\hat{\boldsymbol{d}}} = \operatorname{Proj}_{|\hat{d}_i|\leq D_i}(\Gamma z_2) \tag{8}$$

cancels $z_2^{\mathsf{T}}\tilde{\boldsymbol{d}}$ in (7) and leaves $\dot{V}_a \leq -c\,z_1^{\mathsf{T}}K_1 z_1 - z_2^{\mathsf{T}}K_2 z_2$; the projection keeps $\hat{\boldsymbol{d}}$ bounded without increasing $\dot{V}_a$ when $|d_i| \leq D_i$ [4, 1]. This is the thesis's $\dot{\hat{f}} = -\Gamma^{-1}z$ law with a bound added. Here $\Gamma_i = 0.3\cdot 2J_i\omega_i^2/k_{1,i}$, which makes the integral action on z_1 ($\Gamma k_1 \int z_1$) equal to the PID family's, and $D_i = 0.25\,\ell_i$.

Rate transport. $\bar{\boldsymbol{\omega}}_d = R(q_e)^\mathsf{T}\boldsymbol{\omega}_d$ and, since q_e rotates with body rate $\boldsymbol{\omega}_e$ ($\dot{R}_e = R_e[\boldsymbol{\omega}_e]_\times$),

$$\dot{\bar{\boldsymbol{\omega}}}_d = R(q_e)^\mathsf{T}\dot{\boldsymbol{\omega}}_d - \boldsymbol{\omega}_e \times \bar{\boldsymbol{\omega}}_d,$$

with $\boldsymbol{\omega}_d$, $\dot{\boldsymbol{\omega}}_d$ the same central differences as Sec. 2.2. Unlike the A5 families, this law uses the transport, the gyroscopic term and $\dot{\boldsymbol{\omega}}_d$ by construction; the `--no-model-ff` flag does not apply.

4.4 Gain design: same linearization as PD+ff

About $z = 0$, (5)–(6) give, per axis, $\ddot{z}_{1,i} + (k_{1,i}/2 + k_{2,i}/J_i)\dot{z}_{1,i} + (c + k_{1,i}k_{2,i})/(2J_i)\, z_{1,i} = 0$. Writing $k_{1,i} = 2\lambda_{1,i}$, $k_{2,i} = J_i\lambda_{2,i}$ and imposing

$$\lambda_{1,i} + \lambda_{2,i} = 2\zeta\omega_i, \qquad \lambda_{1,i}\lambda_{2,i} + \frac{c}{2J_i} = \omega_i^2, \qquad \omega_i = \rho\omega_{n,i},\ \zeta = 0.95,$$

makes the small-error poles exactly those of the paper's PD+ff ($-11.40 \pm 3.75j$ on roll). Any difference between the two families is then due to the backstepping structure, not to bandwidth or damping. V_1 is a scalar, so one c serves all axes; real λ's need $c \geq 2J_i\omega_i^2(1-\zeta^2)$ on every axis, and the smallest such value is used ($\lambda_1 \leq \lambda_2$: the outer kinematic loop is the slower one). For X2: $c = 1.705\,\mathrm{N\,m}$, $\boldsymbol{k}_1 = (22.80, 16.69, 10.40)\,\mathrm{s}^{-1}$, $\boldsymbol{k}_2 = (0.692, 0.527, 0.375)\,\mathrm{N\,m\,s}$.

4.5 Verification

`verify_true_backstepping.py` checks the derivation on the rigid-body model (`results/verification.json`). *Verified*:

- $\dot{z}_1$ and $\dot{\bar{\boldsymbol{\omega}}}_d$ in closed form against central finite differences along rigid-body trajectories on Full3, Hard45 and Yaw360: worst absolute error 5.6×10^{-7} (signals of order 0.1–1);
- closed loop, plain law, no torque limit, Full3 and Hard45 from a $\approx 20°$ initial error: V decreases monotonically to 10^{-12}, and the finite-difference $\dot{V}$ equals $-c\, z_1^\mathsf{T}K_1z_1 - z_2^\mathsf{T}K_2z_2$ within 0.22% (relative, step-size limited);
- adaptive law with a constant unknown torque of 20% of the box: $\hat{\boldsymbol{d}}$ reaches $\boldsymbol{d}$ within 0.25% after 20 s;
- linearized poles equal the PD+ff poles to 2.7×10^{-15}.

`experiments/backstepping_true_2026-10-01/true_backstepping.py`: 44–55 gain design: smallest common c, $\lambda_1 \leq \lambda_2$

```
44 def design_gains(J: np.ndarray, w: np.ndarray, zeta: float):
45     """Return k1 (1/s), k2 (N m s), c (N m) and lam1, lam2 per axis (see module docstring)."""
46     J, w = np.asarray(J, float), np.asarray(w, float)
47     c = float(np.max(2.0 * J * w ** 2 * (1.0 - zeta ** 2)))          # smallest c with real lam1, lam2 on all
       axes
48     if np.any(c > 2.0 * J * w ** 2):
49         raise ValueError("no positive lam1*lam2 for this inertia spread; change zeta")
50     disc = np.sqrt(np.maximum((zeta * w) ** 2 - w ** 2 + c / (2.0 * J), 0.0))
51     lam1, lam2 = zeta * w - disc, zeta * w + disc
52     return 2.0 * lam1, J * lam2, c, lam1, lam2
53
54
55 class TrueBackstepping:
```

`experiments/backstepping_true_2026-10-01/true_backstepping.py` : 84–114 z_1, z_2, $\dot{z}_1$, transport, law (6), projected adaptation (8)

```python
    def signals(self, t, q_meas, omega):
        """Backstepping variables at (t, q, w): z1, z2, w_e, z1_dot, wbar_d, wbar_d_dot, |q_e0|."""
        from gra_control.math3d import reference_quat
        qd = reference_quat(t, self.mission, self.duration)
        qe, z1 = quat_error(qd, q_meas)
        a0 = abs(qe[0])
        wd, wd_dot = self._wd(t), self._wd_dot(t)
        Rt = rot(qe).T
        wbar = Rt @ wd
        we = omega - wbar
        z1_dot = 0.5 * (a0 * we + np.cross(z1, we))
        wbar_dot = Rt @ wd_dot - np.cross(we, wbar)
        z2 = we + self.k1 * z1
        return z1, z2, we, z1_dot, wbar, wbar_dot, a0

    def torque(self, t, eq, omega, q_meas):
        z1, z2, we, z1_dot, wbar, wbar_dot, a0 = self.signals(t, q_meas, omega)
        tau = (np.cross(omega, self.J * omega) + self.J * (wbar_dot - self.k1 * z1_dot)
               - self.c * z1 - self.k2 * z2)
        if self.adaptive:
            tau = tau - self.d_hat
            step = self.G * z2 * self.dt
            new = self.d_hat + step
            clipped = np.clip(new, -self.D, self.D)          # projection onto the box |d_hat_i| <= D_i
            self.n_proj += int(np.any(clipped != new))
            self.d_hat = clipped
        self.n += 1
        if self.n % 200 == 1:
            V = self.c * 2.0 * (1.0 - a0) + 0.5 * float(np.sum(self.J * z2 ** 2))
            self.V_log.append(V)
        return np.clip(tau, -self.lim, self.lim)
```

`experiments/backstepping_true_2026-10-01/run_true_backstepping.py` : 30–42 the family enters the unchanged A5 runner

```python
_Original = R.AdvancedBaseline


def _factory(args, inertia, limits, wn):
    if args.baseline in NEW:
        return TrueBackstepping(inertia, limits, wn, R.reference_rate, args.mission, args.duration,
                                rho=float(args.bw_scale), zeta=R.BASELINE_ZETA,
                                adaptive=args.baseline == "bs-adaptive", dt=args.dt, rate_ff=args.rate_ff)
    return _Original(args, inertia, limits, wn)


R.AdvancedBaseline = _factory
R.BASELINE_FAMILIES = R.BASELINE_FAMILIES + NEW
```

Implementation note. The Lyapunov certificate holds for the model (5) without saturation. In the plant the torque box, rotor clipping, the non-diagonal X2 inertia (the law uses the principal moments, as every family does) and the residual add terms that the certificate does not cover; the sign selection makes V_1 non-smooth at $q_{e,0} = 0$, the same local scope as the paper. The output is saturated to the box, so Proposition 1 applies unchanged.

4.6 Results: recursive backstepping with and without the residual

Same protocol as A5 (Table 7): 45-s missions, $\rho = 3$, noise-free, seed 1, gate on, the residual unchanged. Run by `experiments/backstepping_true_2026-10-01/run_campaign_true_bs.py`; numbers computed from the run summaries by `tools/build_family_tables.py`.

Table 4: Reduction of mean quaternion-angle error by the gated CMG–PES residual on the recursive backstepping laws (%, positive is better; 5-mission = reduction of geometric means; last column: baseline geometric mean, mrad).

Family	Platform	Roll	Full3	Hard45	Hardstep	Yaw360	5-mission	base (mrad)
Recursive backstepping	X2	-114.8	+7.2	+11.8	-2.9	+43.5	**-0.4**	3.30
	CF2	-1416.5	-18.2	-38.1	-0.8	-1467.9	**-230.0**	0.59
Adaptive recursive backstepping	X2	-114.7	+6.5	+11.7	-0.8	+30.9	**-4.3**	2.63
	CF2	-669.0	-9.6	-19.6	-0.4	-2494.5	**-204.7**	0.88

Table 5: Nominal laws alone (no residual): error reduction of the recursive designs relative to the computed-torque PD+ff (the paper's primary baseline), % (positive: the recursive law tracks better). The linearizations are identical, so these differences come from the backstepping structure.

Platform	law	Roll	Full3	Hard45	Hardstep	Yaw360	5-mission
X2	recursive	+29.6	+30.5	+31.0	-2.4	+41.5	**+27.4**
X2	adaptive	+36.5	+30.8	+29.1	-7.9	+80.7	**+42.2**
CF2	recursive	+95.0	+64.8	+73.3	-0.5	+97.9	**+84.2**
CF2	adaptive	+91.2	+47.5	+58.9	-695.8	+99.6	**+76.7**

What the two tables show.

- **The recursive law is a much better nominal controller** (Table 5): with the same linearization it reduces the computed-torque PD+ff error on four of five missions on both platforms, most strongly on CF2. Its law contains $\boldsymbol{J}\dot{\boldsymbol{\omega}}_d$ and $\boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}$, which every A5 family ran without (`--no-model-ff`). The computed-torque law *with* model feedforward was not run, so this campaign does not separate the share of the gain due to the acceleration feedforward from the share due to the recursive structure (cross term and rate transport). That run is the natural next control.

- **The unchanged residual does not help it** (Table 4). It still reduces the error on X2 Full3, Hard45 and Yaw360, but where the recursive law's error is already very small (Roll; CF2 Roll and Yaw360, a few hundredths of a mrad) it raises the error to tenths of a mrad, worse in absolute terms than the residual on the paper's baseline. The residual's β, η and k_e were set for a lagging PD+ff; on a nominal law that already cancels the dominant error, the residual's own correction becomes the error. Retuning (β in particular) was not tested. This is a limit of the unchanged residual, not of Proposition 1, which still holds: the output stays in the box and $\beta = 0$ recovers the law.

- **The adaptive variant fails on CF2 Hardstep** under persistent rotor saturation (79% of samples): the projected estimate keeps integrating z_2 while the torque it requests cannot be delivered, a windup through the adaptation law that the Lyapunov argument (which assumes no saturation) does not exclude. Freezing the adaptation when the box or a rotor saturates, the rule the PID family uses, is the standard fix and was not run. On the other missions the adaptive and plain laws are close.

5 PID with rate feedforward and rotor-level anti-windup

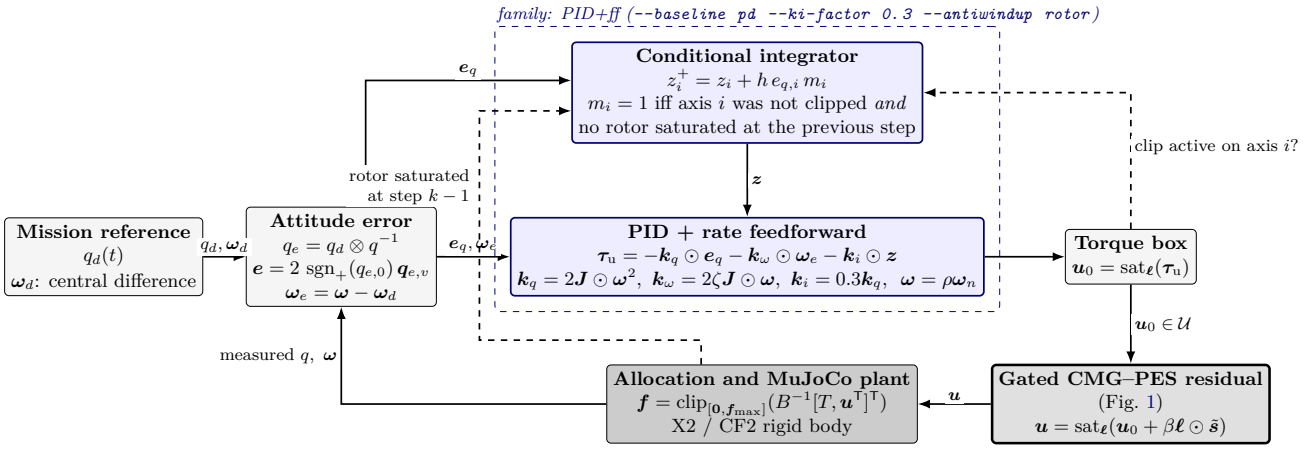


Figure 4: PID+ff. The integrator runs on $\boldsymbol{e}_q$ and is frozen per axis whenever the torque box clipped that axis, and on all axes whenever any rotor saturated at the previous step (dashed paths).

Law. The PID path bypasses `AdvancedBaseline` (`--baseline pd`) and adds an integral term to P-(15):

$$\boldsymbol{u}_0 = \mathrm{sat}_{\boldsymbol{\ell}}\big(-\boldsymbol{k}_q \odot \boldsymbol{e}_q - \boldsymbol{k}_\omega \odot \boldsymbol{\omega}_e - \boldsymbol{k}_i \odot \boldsymbol{z}\big), \qquad \boldsymbol{k}_q = 2\boldsymbol{J} \odot (\rho\boldsymbol{\omega}_n)^2,\ \boldsymbol{k}_\omega = 2\zeta \boldsymbol{J} \odot \rho\boldsymbol{\omega}_n,\ \boldsymbol{k}_i = 0.3\,\boldsymbol{k}_q. \quad (9)$$

Conditional integration. With $h = 5 \times 10^{-4}\,\mathrm{s}$ and the clip test of line 357,

$$z_{i,k+1} = z_{i,k} + h\, e_{q,i,k}\, m_{i,k}, \qquad m_{i,k} = \mathbf{1}\big[u_{0,i,k} = \tau_{\mathrm{u},i,k}\big] \cdot \mathbf{1}\big[\text{no rotor saturated at } k-1\big]. \quad (10)$$

This is conditional integration (integrator clamping) [5], applied at the rotor level as well as at the torque box: a rotor at its force limit means the realized torque differs from the command even if the box is inactive. *Standard; the rotor-level rule is this study's choice.*

Loop. In the coordinate $\boldsymbol{e}$ the gains per unit inertia are $k_1 = (\rho\omega_n)^2$, $k_2 = 2\zeta\rho\omega_n$ and $k_I = 0.3(\rho\omega_n)^2$ (Table 2). The Routh condition for $s^3 + k_2 s^2 + k_1 s + k_I$ is $k_2 k_1 > k_I$, met with a ratio of at least 66.5. The integral adds a slow real pole at $-0.32\,\mathrm{s}^{-1}$ (time constant $\approx 3\,\mathrm{s}$, Table 3): the integrator is weak, which the A5 report discloses as "undertuned". *Derived here; verified.*

`experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py`: 253–260 gains; PID uses the plain PD path

```
253     wn_pd = BASE_WN_RAD_S * args.bw_scale
254     kq_pd = 2.0 * inertia * wn_pd * wn_pd
255     kw_pd = 2.0 * BASELINE_ZETA * wn_pd * inertia
256     ki_pd = args.ki_factor * kq_pd
257     eq_integral = np.zeros(3)
258     integral_frozen_steps = np.zeros(3, dtype=int)
259     rotor_sat_prev = False
260     baseline = AdvancedBaseline(args, inertia, limits, BASE_WN_RAD_S) if args.baseline != "pd" else None
```

`experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py` : 347–361 (9) and the conditional integrator (10)

```
            if args.baseline != "pd":
                tau0 = baseline.torque(t, eq_feedback, omega_feedback, q_feedback)
            else:
                omega_d = (np.zeros(3) if args.rate_ff == "none" else
                           reference_rate(t, args.mission, args.duration, args.rate_ff))
                tau0_unsat = -kq_pd * eq_feedback - kw_pd * (omega_feedback - omega_d) - ki_pd * eq_integral
                tau0 = np.clip(tau0_unsat, -limits, limits)
                if args.ki_factor > 0.0:
                    # Conditional-integration anti-windup (rotor-level: freeze when last step saturated a
                    # rotor, not just the torque box -- CONTROLLER_EXTENSION_DECISION.md's stated requirement).
                    free = np.isclose(tau0, tau0_unsat, rtol=0.0, atol=1e-12)
                    if args.antiwindup == "rotor" and rotor_sat_prev:
                        free = np.zeros(3, dtype=bool)
                    eq_integral = eq_integral + np.where(free, eq_feedback, 0.0) * args.dt
                    integral_frozen_steps = integral_frozen_steps + (~free).astype(int)
```

`experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py` : 475–475 rotor-saturation flag for the next step

```
            rotor_sat_prev = bool(np.any(rotor_sat_mask))
```

Implementation note. The integrator acts on $\boldsymbol{e}_q$ (not $\boldsymbol{e} = 2\boldsymbol{e}_q$), so $\boldsymbol{k}_i = 0.3\boldsymbol{k}_q$ means $k_I = 0.3(\rho\omega_n)^2$ in Table 2's coordinate. With `--ki-factor 0` this path is P-(15) exactly.

6 LQR

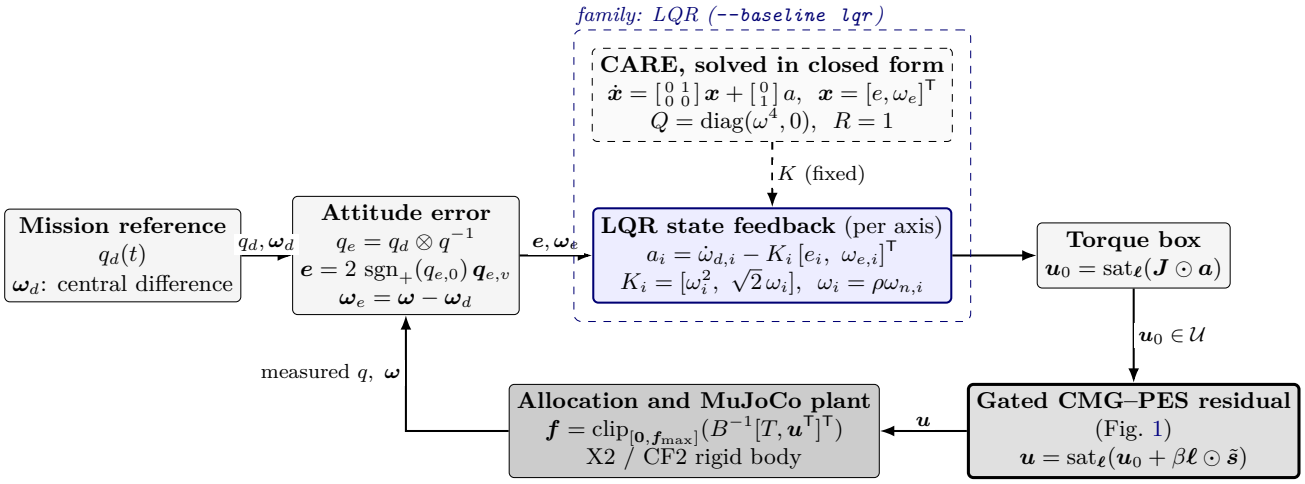


Figure 5: LQR. The gain is computed once from the closed-form solution of the continuous algebraic Riccati equation (CARE) for the per-axis double integrator; online it is a fixed state feedback.

Design problem. For each axis, $\boldsymbol{x} = [e, \omega_e]^\mathsf{T}$, $\dot{\boldsymbol{x}} = A\boldsymbol{x} + Ba$ with $A = \left[\begin{smallmatrix}0&1\\0&0\end{smallmatrix}\right]$, $B = [0, 1]^\mathsf{T}$, minimize $\int_0^\infty (q_e e^2 + r a^2)\, dt$ [6].

Closed-form CARE. With $P = \left[\begin{smallmatrix}p_1&p_2\\p_2&p_3\end{smallmatrix}\right]$, the CARE $A^\mathsf{T}P + PA - PBr^{-1}B^\mathsf{T}P + Q = 0$ reads, entry by entry,

$$(1,1):\ q_e - \frac{p_2^2}{r} = 0, \qquad (1,2):\ p_1 - \frac{p_2 p_3}{r} = 0, \qquad (2,2):\ 2p_2 - \frac{p_3^2}{r} = 0.$$

The positive-definite solution has $p_2 = \sqrt{q_e r}$, $p_3 = \sqrt{2p_2 r}$, $p_1 = p_2 p_3 / r$, so

$$K = r^{-1}B^\mathsf{T}P = \left[\sqrt{q_e/r},\ \sqrt{2\sqrt{q_e/r}}\right] \xrightarrow{q_e/r = \omega^4} K = [\omega^2,\ \sqrt{2}\,\omega], \tag{11}$$

a closed loop $s^2 + \sqrt{2}\omega s + \omega^2$ with $\zeta = 1/\sqrt{2}$ at the same $\omega = \rho\omega_n$ as the other families. *Derived here.* *Verified*: P from these formulas leaves a CARE residual of 2.7e-18 (relative), its smallest eigenvalue is $7.39 > 0$, and $B^\mathsf{T}P$ equals the code's gain.

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 132–137 $K = [\omega^2, \sqrt{2}\omega]$

```
132         if b == "lqr":
133             # Per-axis double integrator x = [e, w_e], input = angular accel.
134             # With Q = diag(q_e, 0), R = r the CARE has the closed form
135             # K = [sqrt(q_e/r), sqrt(2 sqrt(q_e/r))]; choosing q_e/r = wn^4
136             # gives K = [wn^2, sqrt(2) wn], i.e. zeta = 1/sqrt(2).
137             self.k1, self.k2 = self.wn ** 2, np.sqrt(2.0) * self.wn
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 232–233 the law: $\boldsymbol{a} = \dot{\boldsymbol{\omega}}_d - k_1\boldsymbol{e} - k_2\boldsymbol{\omega}_e$

```
232         if b in ("lqr", "backstepping"):
233             acc = wd_dot - self.k1 * e - self.k2 * we
```

7 Sliding-mode control

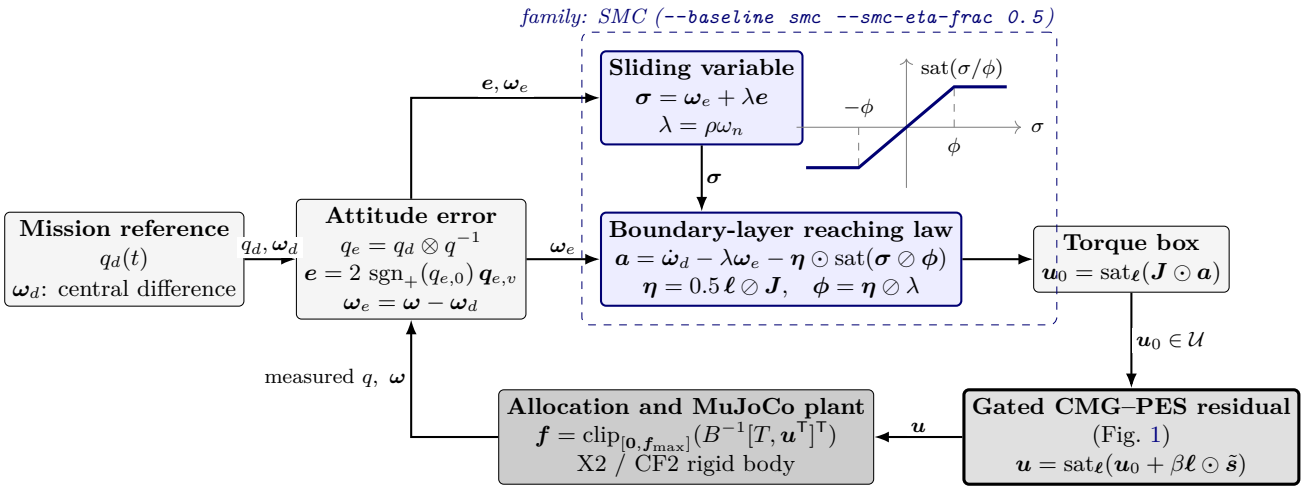


Figure 6: SMC. A linear sliding variable and a boundary-layer reaching law; inside the layer the law is a critically damped PD, outside it the reaching term has a fixed magnitude of half the torque box.

Law. Sliding variable and reaching law [7, 2], per axis:

$$\sigma_i = \omega_{e,i} + \lambda_i e_i, \qquad a_i = \dot{\omega}_{d,i} - \lambda_i \omega_{e,i} - \eta_i\ \mathrm{sat}(\sigma_i/\phi_i), \qquad \lambda_i = \rho\omega_{n,i},\quad \eta_i = 0.5\,\ell_i/J_i,\quad \phi_i = \eta_i/\lambda_i. \tag{12}$$

With the A5 setting, $\boldsymbol{\eta} = (6.00, 6.00, 5.00)\,\mathrm{rad/s^2}$ and $\boldsymbol{\phi} = (0.500, 0.500, 0.476)\,\mathrm{rad/s}$ on both platforms.

Inside the boundary layer ($|\sigma_i| < \phi_i$): $a_i = -\lambda_i\omega_{e,i} - (\eta_i/\phi_i)(\omega_{e,i} + \lambda_i e_i)$, and $\eta_i/\phi_i = \lambda_i$, so

$$a_i = -\lambda_i^2 e_i - 2\lambda_i\omega_{e,i}: \qquad k_1 = \lambda^2, \quad k_2 = 2\lambda, \quad \zeta = 1,$$

a double pole at $-\rho\omega_n$ (Table 3). *Derived here; verified from the code's* `lam`, `eta`, `phi`.

Outside the layer. On the design model (2) with model feedforward off, $\dot{\sigma}_i = \dot{\omega}_{e,i} + \lambda_i\dot{e}_i = a_i - \dot{\omega}_{d,i} + \lambda_i\omega_{e,i} = -\eta_i\,\mathrm{sgn}(\sigma_i) - \dot{\omega}_{d,i}$, hence

$$\sigma_i\dot{\sigma}_i \le -(\eta_i - |\dot{\omega}_{d,i}|)\,|\sigma_i|,$$

the reaching condition, whenever the reference acceleration stays below η_i ($6\,\mathrm{rad/s^2}$ on roll and pitch). The layer is then reached in finite time and the error decays inside it; the boundary layer trades exact sliding for a continuous, chattering-free command. Because $J_i\eta_i = \ell_i/2$, the reaching term alone uses half the torque box, leaving the other half for $-\lambda\boldsymbol{\omega}_e$. *Standard argument, specialized here.*

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 142–149 $\lambda = \omega$, $\eta = 0.5\,\boldsymbol{\ell} \oslash \boldsymbol{J}$, $\phi = \eta/\omega$

```
        elif b == "smc":
            # s = w_e + lam e; accel = -lam w_e - eta sat(s/phi). Inside the
            # boundary layer this is a PD with k1 = lam eta/phi, k2 = lam +
            # eta/phi; lam = wn and eta/phi = wn make it critically damped at
            # wn, outside it a bounded reaching law of magnitude eta.
            self.lam = self.wn.copy()
            self.eta = float(args.smc_eta_frac) * self.lim / self.J
            self.phi = self.eta / self.wn
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 234–236 (12)

```
        elif b == "smc":
            s_ = we + self.lam * e
            acc = wd_dot - self.lam * we - self.eta * np.clip(s_ / self.phi, -1.0, 1.0)
```

8 Model predictive control (unconstrained, finite horizon)

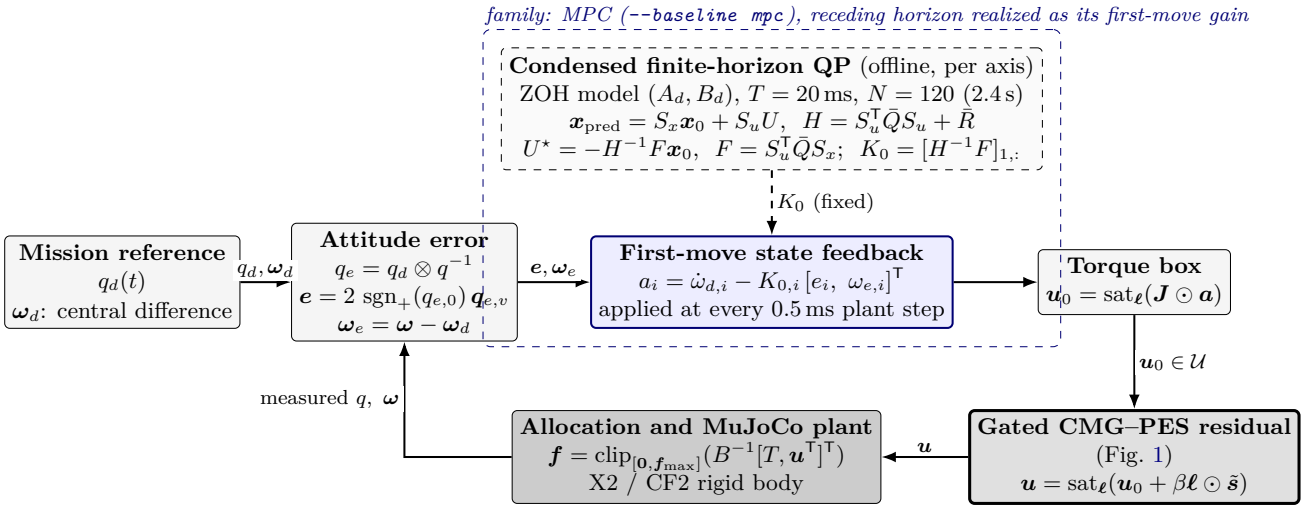


Figure 7: MPC. The finite-horizon quadratic program has no constraints, so its solution is linear in the state; the first move of the optimal sequence is a fixed gain K_0, computed once and applied at every plant step.

Prediction model. Per axis, the exact zero-order-hold discretization of the double integrator with sample time $T = 20\,\mathrm{ms}$,

$$\boldsymbol{x}_{k+1} = A_d\boldsymbol{x}_k + B_d a_k, \qquad A_d = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix}, \quad B_d = \begin{bmatrix} T^2/2 \\ T \end{bmatrix}.$$

Condensed problem. Stacking $N = 120$ predictions, $X = S_x\boldsymbol{x}_0 + S_uU$ with $S_x = [A_d; A_d^2; \ldots; A_d^N]$ and the block-Toeplitz S_u with blocks $A_d^{k-j}B_d$ ($j \le k$). The cost $\sum_{k=1}^{N} \boldsymbol{x}_k^{\mathsf{T}}Q\boldsymbol{x}_k + \sum_{k=0}^{N-1} Ra_k^2$ with $Q = \mathrm{diag}(\omega^4, q_\omega\omega^2)$, $q_\omega = 0$, $R = 1$ becomes

$$U^{\mathsf{T}}HU + 2\boldsymbol{x}_0^{\mathsf{T}}F^{\mathsf{T}}U + \mathrm{const}, \qquad H = S_u^{\mathsf{T}}\bar{Q}S_u + \bar{R}, \quad F = S_u^{\mathsf{T}}\bar{Q}S_x,$$

with $\bar{Q} = I_N \otimes Q$, $\bar{R} = I_N \otimes R$. Without constraints the minimizer is $U^\star = -H^{-1}F\boldsymbol{x}_0$, and the receding-horizon law applies its first element, $a_0 = -K_0\boldsymbol{x}_0$ with $K_0 = [H^{-1}F]_{1,:}$ [8, 9]. *Standard.*

What the horizon does. Table 6 recomputes K_0 with the code's own function for growing N. From $N \approx 30$ (0.6 s, several closed-loop time constants) the first-move gain no longer changes: at the A5 horizon it equals the infinite-horizon discrete LQR gain obtained by iterating the Riccati recursion to its fixed point to a relative 4.6e-10 ($K_0 = [121.55,\ 15.592]$ on roll; DARE residual 9.5e-14). The MPC row is therefore a discrete LQR designed at $T = 20$ ms. That sample time, not the horizon, is why its proportional gain is 16% below the continuous LQR of Sec. 6 (121.5 vs. 144 s^{-2}). *Verified.*

Table 6: First-move MPC gain on the roll axis ($\omega = 12$ rad/s) versus horizon, from `AdvancedBaseline._mpc_first_gain`; the last two columns divide by the continuous LQR gain $[\omega^2, \sqrt{2}\omega]$. A5 uses $N = 120$.

N	NT (s)	$K_{0,1}$	$K_{0,2}$	$K_{0,1}/\omega^2$	$K_{0,2}/(\sqrt{2}\omega)$
5	0.10	84.89	6.388	0.589	0.376
15	0.30	117.86	15.343	0.818	0.904
30	0.60	121.54	15.591	0.844	0.919
60	1.20	121.55	15.592	0.844	0.919
120	2.40	121.55	15.592	0.844	0.919
240	4.80	121.55	15.592	0.844	0.919

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 167–187 condensed matrices S_x, S_u, H, F and the first-move gain

```
    # ---- MPC: aero2's unconstrained finite-horizon first-move gain ---------
    def _mpc_first_gain(self, i):
        T, N = float(self.a.mpc_dt), int(self.a.mpc_horizon)
        Ad = np.array([[1.0, T], [0.0, 1.0]])
        Bd = np.array([[0.5 * T * T], [T]])          # exact ZOH, accel input
        Q = np.diag([self.wn[i] ** 4, float(self.a.mpc_q_rate) * self.wn[i] ** 2])
        R = np.array([[1.0]])
        n, m = 2, 1
        Sx = np.zeros((N * n, n))
        Su = np.zeros((N * n, N * m))
        Ap = np.eye(n)
        for k in range(N):
            Ap = Ad @ Ap
            Sx[k * n:(k + 1) * n, :] = Ap
            for j in range(k + 1):
                Su[k * n:(k + 1) * n, j * m:(j + 1) * m] = (
                    np.linalg.matrix_power(Ad, k - j) @ Bd)
        Qb, Rb = np.kron(np.eye(N), Q), np.kron(np.eye(N), R)
        H = Su.T @ Qb @ Su + Rb
        F = Su.T @ Qb @ Sx
        return np.linalg.solve(H + 1e-9 * np.eye(N * m), F)[:m, :].ravel()
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 237–238 the online law $\boldsymbol{a} = \dot{\boldsymbol{\omega}}_d - K_0[\boldsymbol{e}; \boldsymbol{\omega}_e]$

```
        elif b == "mpc":
            acc = wd_dot - (self.K0[:, 0] * e + self.K0[:, 1] * we)
```

Implementation note. The gain is designed for a 20-ms move but applied as a continuous-time state feedback at every 0.5-ms step (A5 ran every family at full rate). This is a common simplification; it is the reason the MPC row behaves like a softer LQR ($\zeta_{\mathrm{eq}} = 0.71$, $\omega_{\mathrm{eq}} = 11.0$ rad/s) and not like a sampled controller. "MPC" here means an unconstrained receding-horizon gain; the torque box is enforced by the shared clip, not inside the optimization.

9 Nonlinear MPC (grid search around the backstepping law)

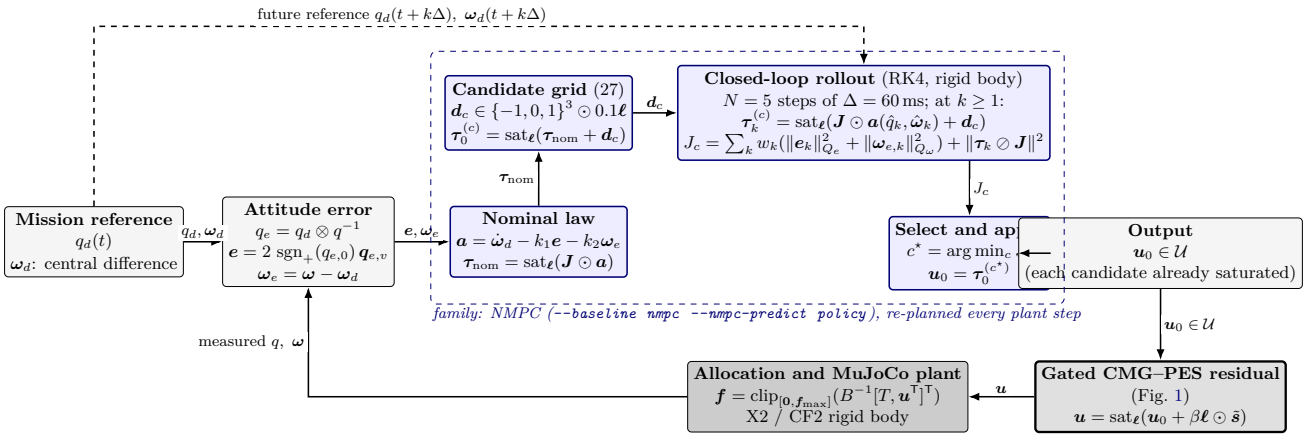


Figure 8: NMPC. At every plant step, 27 candidate offsets are added to the nominal backstepping torque; each candidate is scored by an RK4 rollout of the closed loop on the rigid-body model over 0.3 s, and the first torque of the cheapest candidate is applied.

Candidates. The nominal law is (4) ($\zeta = 0.95$). A grid of $3^3 = 27$ constant offsets $\boldsymbol{d}_c \in \{-1, 0, 1\}^3 \odot 0.1\boldsymbol{\ell}$ is added to it; $\boldsymbol{d}_c = \mathbf{0}$ is one of them (yes), so the nominal law is always a candidate.

Prediction ("policy" mode). From the measured state, each candidate is rolled out for $N = 5$ steps of $\Delta = 60$ ms with the rigid-body model $\dot{Q} = \frac{1}{2}Q \otimes \tilde{\boldsymbol{\omega}}$, $\dot{\boldsymbol{\omega}} = (\boldsymbol{\tau} - \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}) \oslash \boldsymbol{J}$, integrated by classical RK4 [10]. At $k = 0$ the candidate's first torque $\boldsymbol{\tau}_0^{(c)} = \mathrm{sat}_{\boldsymbol{\ell}}(\boldsymbol{\tau}_{\mathrm{nom}} + \boldsymbol{d}_c)$ is used; at $k \geq 1$ the nominal law is re-evaluated on the predicted state and the same offset added, $\boldsymbol{\tau}_k^{(c)} = \mathrm{sat}_{\boldsymbol{\ell}}(\boldsymbol{J} \odot \boldsymbol{a}(\hat{q}_k, \hat{\boldsymbol{\omega}}_k, t + k\Delta) + \boldsymbol{d}_c)$. The prediction thus models the closed loop that will actually run, not a constant torque.

Cost and selection. With $\boldsymbol{e}_k, \boldsymbol{\omega}_{e,k}$ the predicted errors against the reference at $t + (k+1)\Delta$,

$$J_c = \sum_{k=0}^{N-1} \Big[w_k \big(\textstyle\sum_i \omega_i^4 e_{k,i}^2 + 2 \sum_i \omega_i^2 \omega_{e,k,i}^2 \big) + \big\| \boldsymbol{\tau}_k^{(c)} \oslash \boldsymbol{J} \big\|^2 \Big], \qquad w_k = \begin{cases} 1.5, & k = N-1 \\ 1, & \text{otherwise} \end{cases} \tag{13}$$

and $\boldsymbol{u}_0 = \boldsymbol{\tau}_0^{(c^\star)}$, $c^\star = \arg\min_c J_c$. The angle weight ω^4 is the LQR weight q_e/r of Sec. 6; the effort term is the squared commanded acceleration. This is a sampling-based (grid) receding-horizon controller in the sense of [9], not a gradient-based NMPC solver. *Derived here from the code.*

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 152–159 candidate grid and cost weights

```
        if b == "nmpc":
            d = float(args.nmpc_offset_frac) * self.lim
            g = np.linspace(-1.0, 1.0, int(args.nmpc_grid))
            self.offsets = [np.array([i, j, k]) * d for i in g for j in g for k in g]
            self.qe_w = self.wn ** 4          # angle weight, as LQR's q_e/r = wn^4
            # rate-error weight; the metric scores angle only, so a heavy
            # weight here lets the optimiser trade angle for rate matching
            self.qw_w = float(args.nmpc_rate_weight) * self.wn ** 2
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`:201–214 rigid-body model and RK4 step (MuJoCo's body-to-world convention)

```
    # ---- rigid-body prediction in MuJoCo's own convention -------------------
    def _deriv(self, q, w, tau):
        qd = 0.5 * quat_mul(q, np.array([0.0, w[0], w[1], w[2]]))
        wd = (tau - np.cross(w, self.J * w)) / self.J
        return qd, wd

    def _rk4(self, q, w, tau, h):
        k1q, k1w = self._deriv(q, w, tau)
        k2q, k2w = self._deriv(q + 0.5 * h * k1q, w + 0.5 * h * k1w, tau)
        k3q, k3w = self._deriv(q + 0.5 * h * k2q, w + 0.5 * h * k2w, tau)
        k4q, k4w = self._deriv(q + h * k3q, w + h * k3w, tau)
        q = q + (h / 6.0) * (k1q + 2 * k2q + 2 * k3q + k4q)
        w = w + (h / 6.0) * (k1w + 2 * k2w + 2 * k3w + k4w)
        return q / np.linalg.norm(q), w
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`:247–289 `_nmpc`: rollout, cost (13), selection, one-period self-check

```
    def _nmpc(self, t, q_meas, omega, tau_nom):
        a = self.a
        dtp, N = float(a.nmpc_dt), int(a.nmpc_horizon)
        # 'open' (aero2): each candidate holds one constant torque over the
        # whole horizon. Diagnosed 2026-09-10 as the wrong model of a loop
        # that re-plans every period -- it made tracking worse than its own
        # nominal law (0.209 vs 0.118 rad, X2 Full3, 3 s). 'policy': the
        # nominal law is re-evaluated on each predicted state and the
        # candidate adds a constant offset, so the prediction models the
        # closed loop that will actually run.
        tk = [t + k * dtp for k in range(N + 1)]
        qrs = [reference_quat(x, a.mission, a.duration) for x in tk]
        wrs = [self._wd(x) for x in tk]
        wds = [self._wd_dot(x) for x in tk]
        q0 = quat_conj(q_meas)                  # measured -> MuJoCo convention
        policy = a.nmpc_predict == "policy"
        best, best_cost, best_off = tau_nom, np.inf, None
        for off in self.offsets:
            first = np.clip(tau_nom + off, -self.lim, self.lim)
            tau_c = first
            q, w, cost = q0.copy(), omega.copy(), 0.0
            for k in range(N):
                if policy and k > 0:
                    _, ekp = quat_error(qrs[k], quat_conj(q))
                    acc = wds[k] - self.k1 * 2.0 * ekp - self.k2 * (w - wrs[k])
                    g_ = np.cross(w, self.J * w) if self.model_ff else 0.0
                    tau_c = np.clip(self.J * acc + g_ + off, -self.lim, self.lim)
                q, w = self._rk4(q, w, tau_c, dtp)
                _, ek = quat_error(qrs[k + 1], quat_conj(q))
                wek = w - wrs[k + 1]
                wt = float(a.nmpc_terminal) if k == N - 1 else 1.0
                cost += wt * (float(np.sum(self.qe_w * (2.0 * ek) ** 2))
                              + float(np.sum(self.qw_w * wek ** 2)))
                cost += float(np.sum((tau_c / self.J) ** 2))
            if cost < best_cost:
                best, best_cost, best_off = first, cost, off
        self.n_updates += 1
        self.n_nonzero += int(np.any(best_off != 0.0))
        period = self.every * a.dt
        qp, _ = self._rk4(q0.copy(), omega.copy(), best, period)
        self.pred = quat_conj(qp)
        self.q_prev = q_meas.copy()
        return best
```

`experiments/final_robustness_gating_2026-09-20/run_campaign_a5_nmpc_corrected.py`: 25–26 corrected A5 settings: no update hold

```
NMPC_ARGS = ["--nmpc-dt", "0.06", "--nmpc-horizon", "5", "--nmpc-grid", "3", "--nmpc-predict", "policy",
             "--nmpc-offset-frac", "0.1", "--nmpc-rate-weight", "2.0"]   # no --baseline-rate-hz: default 0 = full rate
```

Implementation note. (i) The rollout predicts the nominal family only; it does not know the residual that will be added. (ii) The first A5 NMPC run updated at 100 Hz and held the torque between updates, a leftover from an earlier exploratory setting. The audit (`AUDIT_NMPC_HOLD_RATE_2026-09-25.md`) showed that the hold, not the predictive machinery, caused that row's anomaly: removing it restored NMPC, and imposing the same hold on MPC broke MPC identically. The paper's NMPC row is the full-rate re-run. (iii) The class logs a one-period-ahead self-check of its own prediction (`nmpc_pred_err_mean_deg`); it was used to diagnose (ii), not as a tuning signal.

10 Results across families (Sec. VI-B)

Table 7: Reduction of mean quaternion-angle error by the gated CMG–PES residual on each nominal family, noise-free, $\rho = 3$, 45-s missions (% , positive is better; "5-mission" is the reduction of geometric means). Read from `results/a5_numbers.json`, which `analyze_a5.py` computes from the run summaries; NMPC from the corrected full-rate run. Last column: baseline geometric mean (mrad).

Family	Platform	Roll	Full3	Hard45	Hardstep	Yaw360	5-mission	base (mrad)
PD+ff (backstepping form)	X2	+78.4	+12.4	+13.8	-2.0	+41.8	**+37.3**	4.54
	CF2	+59.9	+5.4	+7.3	-0.2	+20.3	**+22.4**	3.77
PID+ff	X2	+76.8	+14.2	+15.6	-1.2	+40.2	**+36.7**	3.73
	CF2	+59.1	+6.7	+7.7	+2.2	+18.2	**+22.4**	3.13
LQR	X2	+66.3	+12.5	+14.6	-5.5	+41.8	**+31.1**	4.63
	CF2	+59.8	+5.2	+7.7	-0.7	+20.4	**+22.4**	5.77
SMC	X2	+78.4	+16.8	+18.4	+34.2	+41.8	**+43.8**	6.27
	CF2	+59.9	+14.1	+11.8	+12.2	+20.3	**+26.6**	5.34
MPC	X2	+69.7	+14.4	+16.3	-5.2	+42.6	**+33.4**	5.18
	CF2	+59.9	+6.2	+8.5	-1.8	+21.0	**+22.7**	6.40
NMPC	X2	+67.6	+12.5	+13.2	-1.9	+41.8	**+31.9**	4.57
	CF2	+58.7	+7.0	+3.7	+0.1	+20.3	**+21.7**	3.55

These are the numbers the manuscript reports (X2/CF2: backstepping 37.3/22.4, PID+ff 36.7/22.4, LQR 31.1/22.4, SMC 43.8/26.6, MPC 33.4/22.7, NMPC 31.9/21.7), and the claim verifier checks them (`reproduce/verify_acc2027_claims.py`). What the table shows, without further interpretation:

- The five-mission reduction stays within 31–44% on X2 and 22–27% on CF2 across families whose baseline errors differ by up to 1.7× (X2 geometric means 3.7–6.3 mrad), with the residual unchanged.
- The reduction is largest on Roll and Yaw360 and smallest on Hardstep, whose step changes of tens of degrees drive every linear family far past the 5–6° at which its proportional term reaches the box (Table 2).
- SMC is the only family with a clearly positive Hardstep result (+34.2% X2, +12.2% CF2). SMC is also the only family whose law is itself bounded below the box away from the origin (Sec. 7); A5 did not isolate whether that is the cause, so it is recorded here as an open observation.

11 Scope of the family comparison

- The families are *attachment tests*, not tuned competitors: all use the same bandwidth scale $\rho = 3$, model feedforward off, and the A5 settings above. No family was retuned per mission or per platform, and the residual (β, η, k_e) is the same for all.
- MPC is unconstrained and realized as its first-move gain; NMPC is a 27-point grid search around backstepping. Neither is a constrained optimal controller; the torque box acts through the shared saturation.
- PID's integrator is weak (slow pole at $-0.32\,\mathrm{s}^{-1}$) and is disclosed as undertuned in the A5 report.
- All gains are designed on the per-axis double integrator (2) with principal moments; the X2 inertia tensor is not diagonal, so the poles of Table 3 are design values, not measured closed-loop poles.

A Source files and hashes

SHA-256 (first 16 hex digits) of each cited file, compared with the `source_sha256` field of one A5 run per family and platform (Roll, classical arm; NMPC from the corrected run): 60/60 recorded hashes are identical.

File	SHA-256	vs. A5 manifests
`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`	`3c12f663e4034fd9`	identical (12/12 runs)
`experiments/final_robustness_gating_2026-09-20/run_controllers_gated.py`	`48d934309a3f99b8`	identical (12/12 runs)
`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`	`fbfd069d78d4040e`	identical (12/12 runs)
`src/gra_control/math3d.py`	`31620612f184a22f`	identical (12/12 runs)
`src/gra_control/platforms/drone.py`	`50af6d1f6de51788`	identical (12/12 runs)
`experiments/final_robustness_gating_2026-09-20/run_campaign_a5.py`	`1764d6a6be01b967`	campaign driver (not recorded)
`experiments/final_robustness_gating_2026-09-20/run_campaign_a5_nmpc_corrected.py`	`270d9e6fa9f5f96d`	campaign driver (not recorded)
`experiments/final_robustness_gating_2026-09-20/analyze_a5.py`	`c9c994ed338e1b46`	campaign driver (not recorded)
`experiments/backstepping_true_2026-10-01/true_backstepping.py`	`f37226814e9a984f`	new family (Sec. 4)
`experiments/backstepping_true_2026-10-01/run_true_backstepping.py`	`580c71cb16f993d7`	new family (Sec. 4)
`experiments/backstepping_true_2026-10-01/verify_true_backstepping.py`	`b220243925f344f2`	new family (Sec. 4)
`experiments/backstepping_true_2026-10-01/run_campaign_true_bs.py`	`43fed7f09a996bbc`	new family (Sec. 4)

References

[1] M. Krstić, I. Kanellakopoulos, and P. Kokotović, *Nonlinear and Adaptive Control Design*. New York: Wiley, 1995.

[2] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Englewood Cliffs, NJ: Prentice Hall, 1991.

[3] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002.

[4] P. A. Ioannou and J. Sun, *Robust Adaptive Control*. Upper Saddle River, NJ: Prentice Hall, 1996.

[5] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC: ISA – The Instrumentation, Systems, and Automation Society, 2006.

[6] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. Englewood Cliffs, NJ: Prentice Hall, 1990.

[7] V. I. Utkin, *Sliding Modes in Control and Optimization*. Berlin: Springer, 1992.

[8] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge: Cambridge University Press, 2017.

[9] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI: Nob Hill Publishing, 2017.

[10] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Berlin: Springer, 1993.